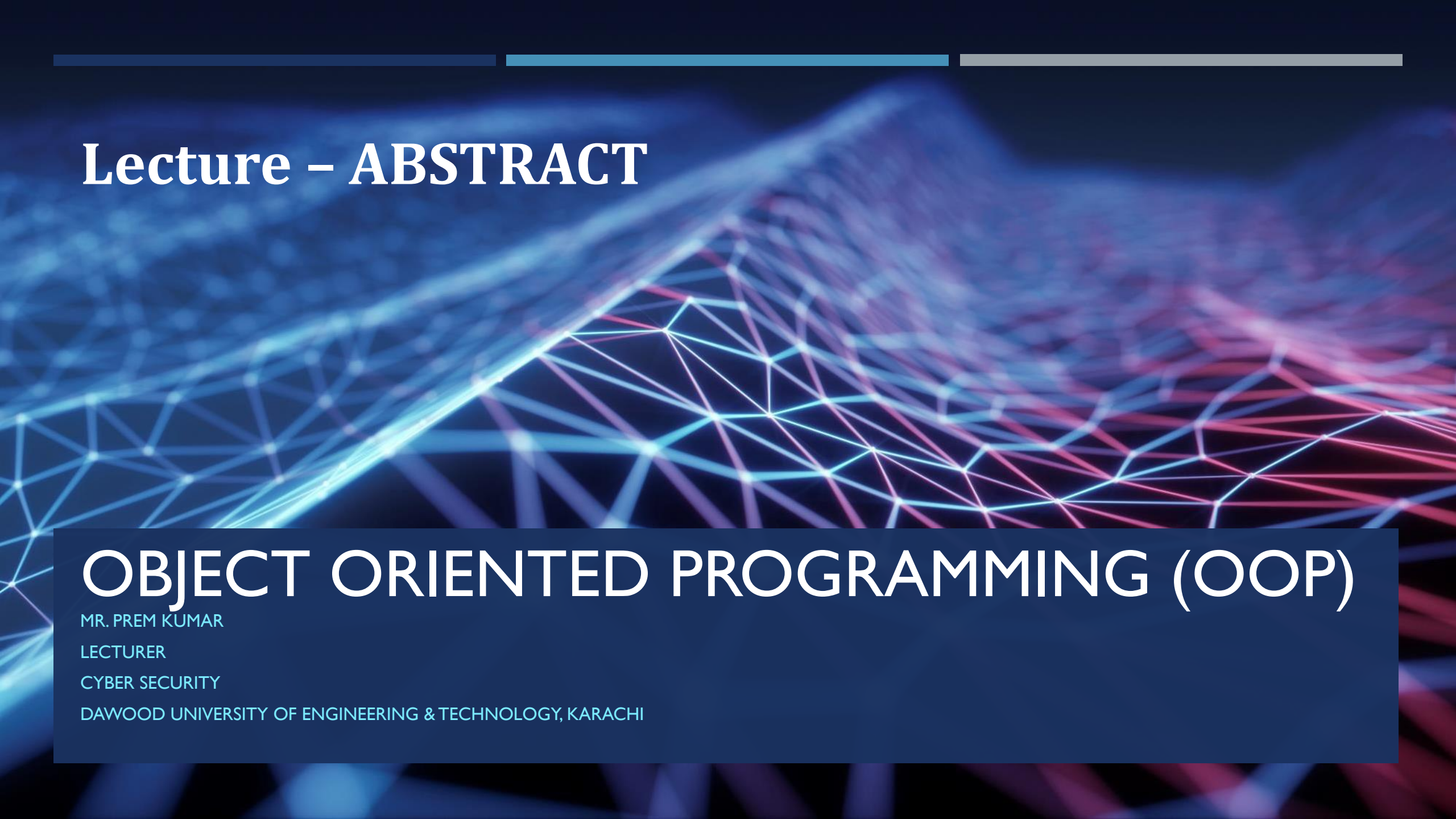




Lecture – ABSTRACT

OBJECT ORIENTED PROGRAMMING (OOP)

MR. PREM KUMAR

LECTURER

CYBER SECURITY

DAWOOD UNIVERSITY OF ENGINEERING & TECHNOLOGY, KARACHI

AGENDA

- INTRODUCTION TO ABSTRACT
- ABSTRACT CLASS
- ABSTRACT METHOD
- EXAMPLES
- toString Method()

ABSTRACT

- According to Google: existing in thought or as an idea but not having a physical or concrete existence.
- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it.

ABSTRACT CLASS IN JAVA

- A class which is declared with the abstract keyword is known as an abstract class in **Java**.
- It can have abstract and non-abstract methods (method with the body).
- Before learning the Java abstract class, let's understand the abstraction in Java first.
- It needs to be extended and its method implemented.
- It cannot be instantiated.

POINTS TO REMEMBER

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Rules for Java Abstract class



ABSTRACT METHODS IN JAVA

- A method which is declared as abstract and does not have implementation is known as an abstract method.

```
abstract class Bike{
    abstract void run();
}
class Honda4 extends Bike{
    void run(){System.out.println("running safely");}
    public static void main(String args[]){
        Bike obj = new Honda4();
        obj.run();
    }
}
```

```
abstract class Shape{
    abstract void draw();
}
//In real scenario, implementation is provided by others i.e. unknown by end user
class Rectangle extends Shape{
    void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape{
    void draw(){System.out.println("drawing circle");}
}
//In real scenario, method is called by programmer or user
class TestAbstraction1{
    public static void main(String args[]){
        Shape s=new Circle1();//In a real scenario, object is provided through method, e.g., getShape() method
        s.draw(); } }
```

//Example of an abstract class that has abstract and non-abstract methods

```
abstract class Bike{  
    Bike(){System.out.println("bike is created");}  
    abstract void run();  
    void changeGear(){System.out.println("gear changed");} }
```

//Creating a Child class which inherits Abstract class

```
class Honda extends Bike{  
    void run(){System.out.println("running safely.");} }
```

//Creating a Test class which calls abstract and non-abstract methods

```
class TestAbstraction2{  
    public static void main(String args[]){  
        Bike obj = new Honda();  
        obj.run();  
        obj.changeGear();  
    }  
}
```





THANK YOU